Lab#01
Structures, Pointers, Arrays

1.1 C++ Structures

C/C++ arrays allow you to define variables that combine several data items of the same kind, but **structure** is another user defined data type which allows you to combine data items of different kinds.

Structures are used to represent a record, suppose you want to keep track of your books in a library. You might want to track the following attributes about each book −

Title

Author

Subject

Book ID

1.1.1 Defining a Structure

To define a structure, you must use the struct statement. The struct statement defines a new data type, with more than one member, for your program. The format of the struct statement is this −

```
struct [structure tag] {

  member definition;

  member definition;

  ...

  member definition;

} [one or more structure variables];
```

The **structure tag** is optional and each member definition is a normal variable definition, such as int i; or float f; or any other valid variable definition. At the end of the structure's definition, before the final semicolon, you can specify one or more structure variables but it is optional. Here is the way you would declare the Book structure −

```
struct Books {

  char  title[50];

  char  author[50];

  char  subject[100];

  int   book_id;

} book;
```

1.1.2 Accessing Structure Members

To access any member of a structure, we use the **member access operator (.)**. The member access operator is coded as a period between the structure variable name and the structure member that we wish to access. You would use **struct** keyword to define variables of structure type. Following is the example to explain usage of structure.

```
#include <iostream>
#include <cstring>

using namespace std;

struct Books {
  char  title[50];
  char  author[50];
  char  subject[100];
  int   book_id;
};

int main() {
  struct Books Book1;       // Declare Book1 of type Book
  struct Books Book2;       // Declare Book2 of type Book

  // book 1 specification
  strcpy( Book1.title, "Learn C++ Programming");
  strcpy( Book1.author, "Chand Miyan");
  strcpy( Book1.subject, "C++ Programming");
  Book1.book_id = 6495407;

  // book 2 specification
  strcpy( Book2.title, "Telecom Billing");
  strcpy( Book2.author, "Yakit Singha");
  strcpy( Book2.subject, "Telecom");
  Book2.book_id = 6495700;

  // Print Book1 info
  cout << "Book 1 title : " << Book1.title <<endl;
  cout << "Book 1 author : " << Book1.author <<endl;
  cout << "Book 1 subject : " << Book1.subject <<endl;
  cout << "Book 1 id : " << Book1.book_id <<endl;

  // Print Book2 info
  cout << "Book 2 title : " << Book2.title <<endl;
  cout << "Book 2 author : " << Book2.author <<endl;
  cout << "Book 2 subject : " << Book2.subject <<endl;
  cout << "Book 2 id : " << Book2.book_id <<endl;

  return 0;
```

```
}
```

When the above code is compiled and executed, it produces the following result –

```
Book 1 title : Learn C++ Programming
Book 1 author : Chand Miyan
Book 1 subject : C++ Programming
Book 1 id : 6495407
Book 2 title : Telecom Billing
Book 2 author : Yakit Singha
Book 2 subject : Telecom
Book 2 id : 6495700
```

1.1.3 Structures as Function Arguments

You can pass a structure as a function argument in very similar way as you pass any other variable or pointer. You would access structure variables in the similar way as you have accessed in the above example –

```
#include <iostream>
#include <cstring>

using namespace std;
void printBook( struct Books book );

struct Books {
  char  title[50];
  char  author[50];
  char  subject[100];
  int   book_id;
};

int main() {
  struct Books Book1;       // Declare Book1 of type Book
  struct Books Book2;       // Declare Book2 of type Book

  // book 1 specification
  strcpy( Book1.title, "Learn C++ Programming");
  strcpy( Book1.author, "Chand Miyan");
  strcpy( Book1.subject, "C++ Programming");
  Book1.book_id = 6495407;

  // book 2 specification
  strcpy( Book2.title, "Telecom Billing");
  strcpy( Book2.author, "Yakit Singha");
  strcpy( Book2.subject, "Telecom");
  Book2.book_id = 6495700;

  // Print Book1 info
```

```
   printBook( Book1 );

   // Print Book2 info
   printBook( Book2 );

   return 0;
}
void printBook( struct Books book ) {
   cout << "Book title : " << book.title <<endl;
   cout << "Book author : " << book.author <<endl;
   cout << "Book subject : " << book.subject <<endl;
   cout << "Book id : " << book.book_id <<endl;
}
```

When the above code is compiled and executed, it produces the following result –

```
Book title : Learn C++ Programming
Book author : Chand Miyan
Book subject : C++ Programming
Book id : 6495407
Book title : Telecom Billing
Book author : Yakit Singha
Book subject : Telecom
Book id : 6495700
```

1.1.4 Pointers to Structures

You can define pointers to structures in very similar way as you define pointer to any other variable as follows –

```
struct Books *struct_pointer;
```

Now, you can store the address of a structure variable in the above defined pointer variable. To find the address of a structure variable, place the & operator before the structure's name as follows –

```
struct_pointer = &Book1;
```

To access the members of a structure using a pointer to that structure, you must use the -> operator as follows –

```
struct_pointer->title;
```

Let us re-write above example using structure pointer, hope this will be easy for you to understand the concept –

```
#include <iostream>
#include <cstring>

using namespace std;
void printBook( struct Books *book );

struct Books {
```

```
   char  title[50];
   char  author[50];
   char  subject[100];
   int   book_id;
};
int main() {
   struct Books Book1;        // Declare Book1 of type Book
   struct Books Book2;        // Declare Book2 of type Book

   // Book 1 specification
   strcpy( Book1.title, "Learn C++ Programming");
   strcpy( Book1.author, "Chand Miyan");
   strcpy( Book1.subject, "C++ Programming");
   Book1.book_id = 6495407;

   // Book 2 specification
   strcpy( Book2.title, "Telecom Billing");
   strcpy( Book2.author, "Yakit Singha");
   strcpy( Book2.subject, "Telecom");
   Book2.book_id = 6495700;

   // Print Book1 info, passing address of structure
   printBook( &Book1 );

   // Print Book1 info, passing address of structure
   printBook( &Book2 );

   return 0;
}

// This function accept pointer to structure as parameter.
void printBook( struct Books *book ) {
   cout << "Book title : " << book->title <<endl;
   cout << "Book author : " << book->author <<endl;
   cout << "Book subject : " << book->subject <<endl;
   cout << "Book id : " << book->book_id <<endl;
}
```

When the above code is compiled and executed, it produces the following result –

```
Book title : Learn C++ Programming
Book author : Chand Miyan
Book subject : C++ Programming
Book id : 6495407
Book title : Telecom Billing
Book author : Yakit Singha
Book subject : Telecom
Book id : 6495700
```

1.1.5 The typedef Keyword

There is an easier way to define structs or you could "alias" types you create. For example –

```
typedef struct {
  char  title[50];
  char  author[50];
  char  subject[100];
  int   book_id;
} Books;
```

Now, you can use *Books* directly to define variables of *Books* type without using struct keyword. Following is the example −

```
Books Book1, Book2;
```

You can use **typedef** keyword for non-structs as well as follows −

```
typedef long int *pint32;


pint32 x, y, z;
```

x, y and z are all pointers to long ints.

1.2 Introduction to Pointers in Data Structure

Pointers are the variables that are used to store the location of value present in the memory. A pointer to a location stores its memory address. The process of obtaining the value stored at a location being referenced by a pointer is known as dereferencing. It is the same as the index for a textbook where each page is referred by its page number present in the index. One can easily find the page using the location referred to there. Such pointers usage helps in the dynamic implementation of various data structures such as stack or list.

1.2.1 Why do We Need Pointers in Data Structure?

Optimization of our code and improving the time complexity of one algorithm. Using pointers helps reduce the time needed by an algorithm to copy data from one place to another. Since it used the memory locations directly, any change made to the value will be reflected at all the locations.

Example:

Call_by_value needs the value of arguments to be copied every time any operation needs to be performed.

Call_by_reference makes this task easier using its memory location to update the value at memory locations.

1.2.1.1 *Control Program Flow*

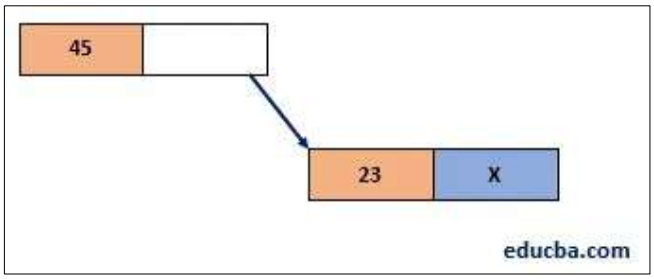
Another use of pointers is to control the program flow. This is implemented by control tables that use these pointers. These pointers are stored in a table to point to each subroutine's entry point to be executed one after the other. These pointers reference the addresses of the various procedures. This helps while working with a recursive procedure or traversal of algorithms where there is a need to store the calling step's location.

1.2.1.2 secondary data structures

secondary data structures such as linked lists or structures to point to the next memory locations in the list.

```
struct Node {
int data;
struct Node* next;
};
```

Example:



1.2.1.3 *Dynamic Memory Allocation*

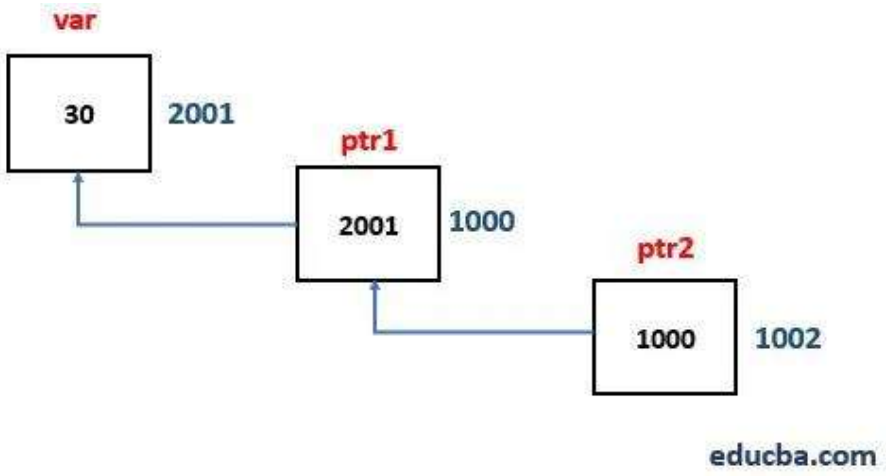
Many programming languages use dynamic memory allocations to allocate the memory for run-time variables. For such type of memory, allocations heap is used rather than the stack, which uses pointers. Here pointers hold the address of these dynamically generated data blocks or array of objects. Many structured or OOPs languages use a heap or free store to provide them with storage locations. The last Node in the linked list is denoted using a NULL pointer that indicates there is no element further in the list.

1.2.2 How do Pointers Work in Data Structure?

Pointers are kind of variables that store the address of a variable.

Defining a Pointer

Here we discuss defining a pointer in the data structure.



Syntax:

```
<datatype> *variable_name
```

Above depicts, vaiable_name is a pointer to a variable of the specified data type.

Example:

```
int *ptr1 - ptr1 references to a memory location
that holds data of int datatype.
int var = 30;
int *ptr1 = &var; // pointer to var
int **ptr2 = & ptr1; // pointer to pointer variable
ptr1
```

In the above example, '&' is used to denote the unary operator, which returns a variable's address.

And '*' is a unary operator that returns the value stored at an address specified by the pointer variable, thus if we need to get the value of variable referenced by a pointer, often called as dereferencing, we use:

```
print("%d", *ptr1) // prints 30

print("%d",**ptr2) // prints 30
```

We need to specify datatype- It helps to identify the number of bytes data stored in a variable; thus. Simultaneously, we increment a pointer variable, and it is incremented according to the size of this datatype only.

C Program on Pointers

Following is an example of creating pointers using C Program.

Example:

```
#include <stdio.h>
void pointerDemo()
{
int var1 = 30;
int *ptr1;
int **ptr2;
ptr1 = &var1;
ptr2 = &ptr1;
printf("Value at ptr1 = %p \n",ptr1);
printf("Value at var1 = %d \n",var1);
printf("Value of variable using *ptr1 = %d \n", *ptr1);
printf("Value at ptr2 = %p \n",ptr2);
printf("Value stored at *ptr2 = %d \n", *ptr2);
printf("Value of variable using  **ptr2 = %d \n", **ptr2);
}
int main()
{
pointerDemo();
return 0;
}
```

Output:

```
Value at ptr1 = 0x7fffc57e1c5c
Value at var1 = 30
Value of variable using *ptr1 = 30
Value at ptr2 = 0x7fffc57e1c60
Value stored at *ptr2 = -981590948
Value of variable using  **ptr2 = 30
```

Explanation: In the above program, we have used single and double dereferencing to display the value of the variable.

There are many types of pointers being **used in computer programming**:

NULL Pointer: Such type of pointer is used to indicate that this points to an invalid object. This type of pointer is often used to represent various conditions such as the end of a list.

VOID Pointer: This type of pointer can be used to point to the address of any type of variable, but the only limitation is that it cannot be dereferenced easily.

WILD Pointer: It is a type of pointer which doesn't hold the address of any variable.

Dangling Pointer: The type of pointers that don't refer to a valid object and are not specifically initialized to point a particular memory. For ex: int *ptr1 = malloc(sizeof(char))

Function pointer: This is a type of pointer to reference an executable code. It is mostly used in the recursive procedure that holds the address of the code that needs to be executed later.

1.2.3 Disadvantage Of Pointers

It can lead to many programming errors as it allows the program to access a variable that has not been defined yet. They can be easily manipulated as the number and made to point some void locations.

Thus to avoid such a situation, many programming languages have started using constructs. Programming languages such as **JAVA has replaced** the concept of pointers with reference variables which can only be used to refer the address of a variable and cannot be manipulated as a number.

1.3 Dynamic Allocation of 2D Arrays in C++ (with code)



You already know that a 2D array array is an array of arrays. But it is defined at the compiled time. In this article, we learn about dynamically allocating a 2D array in C++ at run time. Let's dive deep to understand the dynamic world!

1.3.1 What is a Dynamic 2D Array in C++?

Can you have dynamic arrays in C++? Yes, C++ provides several ways to create dynamic arrays.

We have 2 types of memory: one which we have already before running the program called stack memory and the other is used for dynamic allocation called heap memory. Stack memory is used at compile time of the C++ program while heap memory is used when we use the 'new' operator somewhere in our program.

So, to create a variable or a data structure dynamically, we must use a 'new' operator. When we deal with the heap memory it always returns an address pointing to a created data structure and then we use a pointer to store that address and to access it from heap memory.

Why use heap memory or dynamic allocation? because heap memory is very efficient as it allows us to use only required memory whereas stack memory first comes and then assigns some part of memory to us.

A dynamic array in C++ is an array that can change its size during runtime. It is just like a normal 2D or multi-dimensional array but it is implemented using pointers and memory allocation functions such as new and delete. The main point is that the size of this array is not fixed at compile time.

These arrays are useful when we don't know the size during compilation or we have to change the size during runtime. But be careful while using them as they require extra careful management to avoid memory leaks.

1.3.2 Methods to Dynamically Allocate a 2D Array

Let's now learn about 3 different ways to dynamically allocate a simple 2D array in C++.

1.3.2.1 Single Pointer Method

In this method, a memory block of size M*N is allocated and then the memory blocks are accessed using pointer arithmetic. Below is the program for the same:

```
// C++ program to dynamically allocate
// the memory for 2D array in C++
// using new operator
#include<iostream>
using namespace std;

// Driver Code
int main()
{
	// Dimensions of the 2D array
	int m = 3, n = 4, c = 0;

	// Declare a memory block of
```

```
        // size m*n
        int* arr = new int[m * n];

        // Traverse the 2D array
        for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {

                        // Assign values to
                        // the memory block
                        *(arr + i * n + j) = ++c;
                }
        }

        // Traverse the 2D array
        for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {

                        // Print values of the
                        // memory block
                        cout << *(arr + i * n + j)
                                << " ";
                }
                cout << endl;
        }

        //Delete the array created
        delete[] arr;

        return 0;
}
```

Output:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

First, let's understand this 'new int[m *n]' Here the new operator is used for dynamic allocation and we are creating an array of type int and contiguous allocation of size m * n. This will return us an address as explained above already and stored in a pointer.

Then we are just assigning the values to the address blocks in heap memory using the pointers syntax. 'arr + i * n + j' -> this is the address of the heap block and *(arr + i * n + j) this is the value of that heap block.

1.3.2.2 Using an Array of Pointer

Here an array of pointers is created and then to each memory block. Below is the C++ program to implement it:

```
// C++ program to dynamically allocate
// the memory for 2D array in C++
// using new operator
#include<iostream>
using namespace std;

// Driver Code
int main()
{
	// Dimensions of the array
	int m = 3, n = 4, c = 0;

	// Declare memory block of size M
	int** a = new int*[m];

	for (int i = 0; i < m; i++) {

		// Declare a memory block
		// of size n
		a[i] = new int[n];
	}

	// Traverse the 2D array
	for (int i = 0; i < m; i++) {
		for (int j = 0; j < n; j++) {

			// Assign values to the
			// memory blocks created
			a[i][j] = ++c;
		}
	}

	// Traverse the 2D array
	for (int i = 0; i < m; i++) {
		for (int j = 0; j < n; j++) {

			// Print the values of
			// memory blocks created
			cout << a[i][j] << " ";
		}
		cout << endl;
	}

	// Delete the array created
	for (int i = 0; i < m; i++) // To delete the inner
```

```
                                                        // arrays
            delete[] a[i];
        delete[] a; // To delete the outer array
                            // which contained the pointers
                            // of all the inner arrays

        return 0;
}
```

Output:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

'int a' - > This means we are creating a pointer of type int and this pointer will store the address of an array (in this case) of int type pointers. This array of pointers will point to each row in an array in heap memory.

Practice Problems:

Lab Activity 1:

Write a C++ program of a structure variable to assign data to three members, i-e name, age and salary of a person and display it on screen.

Lab Activity 2: Write a structure to store the roll no., name, age (between 11 and 14) and address of students (more than 10). Store the information of the students.

1 - Write a function to print the names of all the students having age 14.

2 - Write another function to print the names of all the students having even roll no.

3 - Write another function to display the details of the student whose roll no is given (i.e. roll no. entered by the user).

Lab Activity 3:

In C/C++ language, arrays work on memory addresses so as pointers and hence they are strongly related. Design a C program which take an array of 6 elements from the user :and do the following

a) Declare the pointer and refernce it with the array address
b) Displayevery element of an array by using the pointer
c) Find the maximum value in arrayusi n g pointer

```
Enter values into an array:
6
89
56
3
12
5
Display array:
6 89 56 3 12 5
Maximum value is: 89
--------------------------------
Process exited after 11.56 seconds with return value 0
Press any key to continue . . .
```

Lab Activity 4:

Learning Objective: Pointers in function call

In this activity you will have to use pointers to perform the following tasks on randomly generated 1D array of type **int** having size of 20.

i. Write a function **int smallest = smallest_value(int *temp, int size)**, which receives an array as pointer and find out smallest value using pointer to array.

ii. Write a function **void traversal_back(int *temp, int sizeofarry)**, which receives an array as pointer and traverse array from last index to first index and print its value with its address as well.